

# Mamba-Adaptor: State Space Model Adaptor for Visual Recognition

Fei Xie<sup>1</sup> Jiahao Nie<sup>2\*</sup> Yujin Tan<sup>1</sup> Wenkang Zhang<sup>3</sup> Hongshen Zhao<sup>3</sup>

<sup>1</sup> Shanghai Jiao Tong University

<sup>2</sup> Hangzhou Dianzi University

<sup>3</sup> Southeast University

jafffe0319@gmail.com, jhnie@hdu.edu.cn

## Abstract

Recent State Space Models (SSM), especially Mamba, have demonstrated impressive performance in visual modeling and possess superior model efficiency. However, the application of Mamba to visual tasks suffers inferior performance due to three main constraints existing in the sequential model: 1) Casual computing is incapable of accessing global context; 2) Long-range forgetting when computing the current hidden states; 3) Weak spatial structural modeling due to the transformed sequential input. To address these issues, we investigate a simple yet powerful vision task adaptor for Mamba models, which consists of two functional modules: Adaptor-T and Adaptor-S. When solving the hidden states for SSM, we apply a lightweight prediction module Adaptor-T to select a set of learnable locations as memory augmentations to ease long-range forgetting issues. Moreover, we leverage Adaptor-S, composed of multi-scale dilated convolutional kernels, to enhance the spatial modeling and introduce the image inductive bias into the feature output. Both modules can enlarge the context modeling in casual computing, as the output is enhanced by the inaccessible features. We explore three usages of Mamba-Adaptor: A general visual backbone for various vision tasks; A booster module to raise the performance of pretrained backbones; A highly efficient fine-tuning module that adapts the base model for transfer learning tasks. Extensive experiments verify the effectiveness of Mamba-Adaptor in three settings. Notably, our Mamba-Adaptor achieves state-of-the-art performance on the ImageNet and COCO benchmarks.

## 1. Introduction

The architecture of Structured State Space Models (SSMs) [13–15, 38] has gained popularity in language tasks because of their efficiency in modeling long sequences. Re-

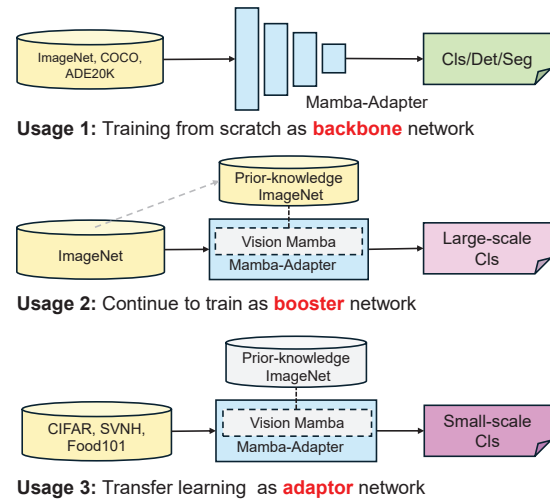


Figure 1. Our Mamba-Adaptor has multiple usages. It can serve as a general visual backbone network, a booster module for pre-trained backbone, and an adaptor for transfer learning.

cently, SSMs, especially the Mamba variant [13], have been applied to vision tasks after the tremendous success of the vision transformers [7, 33, 39, 49, 53, 55, 64, 67]. While the Mamba-based visual models show potential in various downstream tasks [62, 63, 65, 75], such as dense prediction [24, 32, 72, 78], image synthesis [6, 36], and generative tasks [8, 50, 66], the further application and adaptation of Mamba models for the vision field remain largely unexplored.

Despite the superior linear computation complexity with respect to long token length, vision Mamba models have not attained optimal performance in various vision tasks [30, 43, 61, 76] because of three inherent constraints in the SSM mechanism. Firstly, the inherent RNN [20] mechanism of SSM recurrently computes the next time step output using the previously stored memory, also dubbed as hidden states. Thus, the casual computing pipeline is incapable of accessing future output states, while the 2D visual data is a non-casual style. Existing vision Mamba [78] mod-

\*Corresponding author.

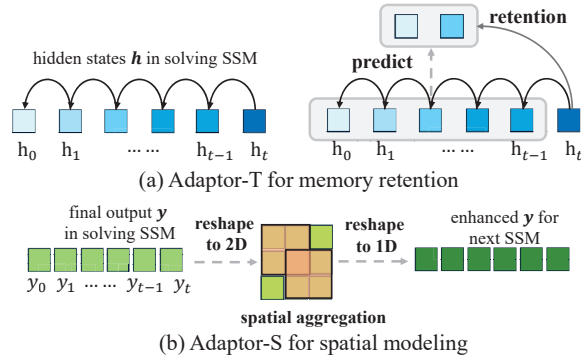


Figure 2. Mamba-Adaptor consists of two modules for solving SSM [13, 44]: Adaptor-T conducts memory retention for the hidden states; Adaptor-S conducts spatial aggregation for final output.

els partially address this issue with a bi-directional scanning strategy, which reverses the sequential order to modeling global context. Secondly, the recurrent computing pipeline in SSM keeps the complexity constant while the influence of the previous hidden state decays as the sequence length increases. Thus, the memory decay results in a long-range forgetting phenomenon, which further weakens the global context modeling capability of the SSM. Thirdly, the current vision Mamba models [32, 78] heavily rely on transforming 2D visual data into 1D sequential format, which destroys the spatial dependencies in structural images. Though dedicated scanning strategies, such as four-way [32] and local windowed scanning [24, 72], can partially preserve the spatial localities, the vision Mamba models still suffer from learning 2D dependencies from the sequence modeling.

After a detailed analysis, we classify the three mentioned constraints into two categories, i.e., temporal decay and spatial locality. Therefore, we introduce a simple yet powerful vision task adapter for Mamba models (shown in Figure 2), which consists of two functional modules: Adaptor-T and Adaptor-S. Adaptor-T conducts memory retention to handle the long-range forgetting issue for each hidden state. It is noteworthy that handcrafted memory selection falls short in finding the most forgetting states, limiting the effects of memory retention. Thus, we apply a lightweight learnable memory selection scheme that dynamically selects a group of the most forgetting states. For spatial modeling enhancement, we directly apply multi-scale depthwise convolution filters to functions at the 2D feature output, which effectively introduces the image inductive bias. Moreover, it is critical to implement our adaptor into the highly optimized Mamba solver. To achieve this, we decouple the solver with identity matrix and highly efficient matrix multiplication, thus Adaptor-S and Adaptor-T can be inserted into Mamba solvers seamlessly. We also devise two insertion forms for Mamba-Adaptor, i.e., parallel form for transfer learning and

sequential form for training from scratch.

As illustrated in Figure 1, our Mamba-Adaptor has multiple usages, including serving as a general vision backbone, a booster module to raise the performance further, and an efficient adapter for transfer learning. Mamba-Adaptor achieves superior performance compared to recent Mamba-based, CNN, and transformer-based methods in classification and dense prediction tasks. Mamba-Adaptor also shows competitive results on various transfer learning tasks. Overall, the contribution is as follows:

- We introduce a plug-and-play Mamba-Adaptor, which can enhance the performance of Mamba models in various vision tasks with acceptable computational overhead.
- Our Mamba-Adaptor consists of two functional modules, Adaptor-T and Adaptor-S, which effectively address the temporal decay and spatial locality issues.
- Mamba-Adaptor not only can serve as a general vision backbone for image classification and prediction tasks but also take effects in transfer learning task settings.
- Extensive experimental results show our Mamba-Adaptor’s superior performance and considerable improvements over Mamba baselines across various vision downstream tasks.

## 2. Related Work

**State Space Model.** State Space Models (SSMs) [13, 15, 38, 44], originating from linear control systems [60], offer an efficient framework with linear computational complexity that makes them well-suited for long-sequence processing. Language tasks [37, 54] are first applied with the SSM-based methods for long-range context modeling. Later, a bunch of improvements, including structural optimizations [15], associative scanings [44], and hardware speed-up techniques [14], are proposed to evolve the SSM with modern deep learning architecture [10, 17, 18, 27, 40, 48, 69, 73]. The recently emerged Mamba [13] introduced input-specific parameterization and parallel scanning (S6), further establishing SSMs as a strong competitor to Transformers. These innovations have led to the widespread application of SSMs in vision tasks, especially as an alternative for transformers [5, 33, 51, 55–57] with superior linear computation complexity. Pioneer vision models such as ViM [78] and VMamba [32] apply the Mamba scheme in visual representation by transforming the 2D images into a sequential format using bidirectional- and cross-scanning strategies. The following vision methods improve the visual capability of Mamba scanning through a more dedicated scanning pattern [24, 41, 68, 72] to preserve spatial locality and hybrid transformer-Mamba architecture [16, 36] as powerful complements. Despite these advances, visual SSM models still face challenges in overcoming the long-range modeling issues and sequential transformation limi-

tations for processing 2D image data, which impedes their performance for vision tasks that require vision-specific priors and capabilities. Our adaptor aims to overcome the shortcomings of long-range forgetting issues and sequence transformation in vision Mamba-based methods.

**Visual Adapter.** The concept of the adaptor originates from the language field [22], which functions as an additional module for fine-tuning large language models for specific purposes [21, 46]. For the vision field, adaptors generally serve as trainable parts to transfer the large-scale knowledge prior to vision foundation models [4, 34, 42, 58], such as CLIP [42], into the various downstream tasks [9, 45, 47, 74]. For transfer learning tasks [22, 25, 59], Adaptorformer [2] uses an additional branch for MLP [52] to finetune the downstream tasks for the pre-trained vision transformer efficiently. VPT [25] adds extra visual tokens as prompt input to fine-tune the base model efficiently. The transfer learning [59] for Mamba models still remains largely unexplored. We develop Mamba-specific fine-tuning modules to adapt Mamba to downstream tasks for transfer learning. Another usage of Adaptors in vision is as a complementary module to enhance the inability of the base model, such as enhancing the hierarchical feature representation in plain vision transformers [7]. Pioneer works like ViTDet [29] implement feature pyramid modules to adapt the plain features extracted from the vision transformer for dense prediction tasks. Another representative work in visual adaptor is ViT-Adaptor [3], which designs a convolutional-based network as a complement branch to the attention-based network, which introduces the necessary inductive bias for image tasks. The Mamba-based architecture [13, 15, 44] is originally designed for language modeling, which neglects the visual priors. We introduce the visual priors and overcome the Mamba constraints, which can serve as a booster module. Our work explores two usages of the Mamba-Adaptor, which serves as a dedicated adaptor design for Mamba models in transfer learning, and a booster module to raise the performance further.

### 3. Method

In this section, we first revisit the preliminaries of the SSM in Section 3.1. Then, we introduce the formulation of the proposed Mamba-Adaptor (Section 3.2) and the two vital submodules: Apdator-T (Section 3.3) and Adaptor-S (Section 3.4). We also depict the practical implementations of the Mamba-Adaptor and how to integrate the adaptor into the Mamba structure in Section 3.5.

#### 3.1. Preliminaries

**State Space Model.** State-space models (SSMs) are often utilized to analyze sequential data and model the input transform for continuous linear time-invariant (LTI) sys-

tems. In SSM, an input sequence  $u(t) \in \mathbb{R}$  is to compute the response  $y(t) \in \mathbb{R}$  by the hidden state  $\mathbf{h}(t) \in \mathbb{R}^N$  as an intermediate value. The process of solving SSM can be expressed as a combination of linear ordinary differential equations (ODEs):

$$\begin{aligned}\mathbf{h}'(t) &= \mathbf{A}\mathbf{h}(t) + \mathbf{B}u(t), \\ y(t) &= \mathbf{C}\mathbf{h}(t) + \mathbf{D}u(t),\end{aligned}\tag{1}$$

where  $\mathbf{A} \in \mathbb{R}^{N \times N}$ ,  $\mathbf{B} \in \mathbb{R}^{N \times 1}$ ,  $\mathbf{C} \in \mathbb{R}^{1 \times N}$ , and  $\mathbf{D} \in \mathbb{R}^1$  are the transforming matrices for the linear state transition and output equations.

**Discretization of SSM.** To effectively solve the SSM [13] in the LTI system within a deep learning framework, it is important to convert the continuous-time equations into a discrete format. A frequently used method in LTI analysis for achieving this is called Zero-Order Hold (ZOH) discretization [28]. A continuous-time system can be converted into a discrete-time system by breaking the input into small time intervals, which can be considered as constant during those intervals. ZOH effectively converts continuous-time parameters ( $\mathbf{A}$ ,  $\mathbf{B}$ ,  $\mathbf{C}$ ,  $\mathbf{D}$ ) in SSM into their discrete equivalents over the fixed sampling time interval  $\Delta \in \mathbb{R}^D$ :

$$\begin{aligned}\bar{\mathbf{A}} &= e^{\Delta\mathbf{A}}, \quad \bar{\mathbf{B}} = (\Delta\mathbf{A})^{-1} (e^{\Delta\mathbf{A}} - \mathbf{I}) \Delta\mathbf{B}, \\ \bar{\mathbf{C}} &= \mathbf{C}, \quad \bar{\mathbf{D}} = \mathbf{D}.\end{aligned}\tag{2}$$

**Selective Scan Mechanism.** To address the limitations of linear time-invariant (LTI) state-space models (SSMs) in capturing contextual information, Mamba [13] introduced a parameterization method featuring an input-dependent selection mechanism called S6 [15]. This allows the model to select relevant features based on specific inputs, enhancing its response to various input contexts. In the context of selective SSMs, the time-varying weighting parameters pose a challenge for efficiently calculating hidden states since convolutions are static and are not designed to handle dynamic weights. Mamba [12] applies an input-dependent scheme to generate the parameters  $\{\mathbf{B}_k\}_{k=1}^L$ ,  $\{\mathbf{C}_k\}_{k=1}^L$  and  $\{\Delta_k\}_{k=1}^L$  from the input sequence  $\{x_k\}_{k=1}^L$ . With the discretization of parameters, the sequential states  $\{h_k\}_{k=1}^L$  can be computed in a recursive manner:

$$h_t = \bar{\mathbf{A}}_t h_{t-1} + \bar{\mathbf{B}}_t u_t,\tag{3}$$

The hidden state  $h_t$  in Eq. 1 can be computed recursively using the previous hidden state  $h_{t-1}$  and current feature input  $u_t$ . Then, we can obtain the final output  $y_t$  from the hidden state:

$$y_t = \mathbf{C}_t h_t + \mathbf{D} u_t.\tag{4}$$

The response  $y_b$  can still be efficiently computed using associative scan algorithms [44] with linear complexity. Hardware optimization tricks are also applied to speed up solving SSM [13, 15].

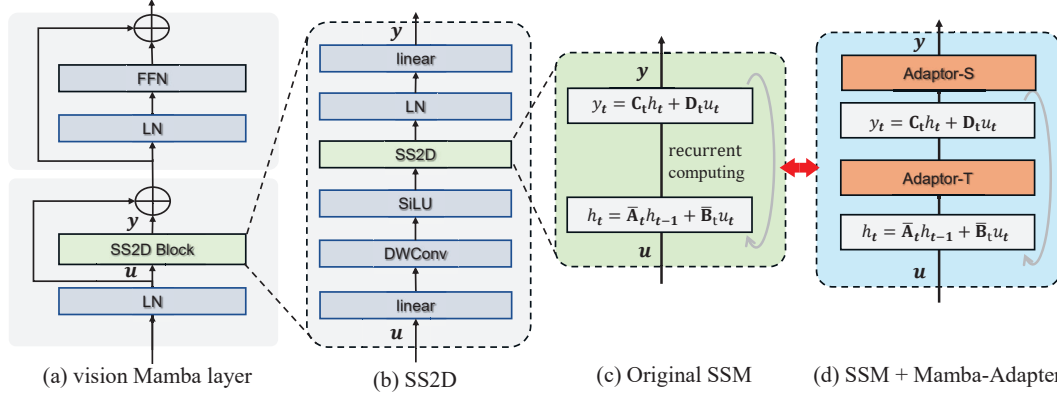


Figure 3. Details of the Mamba-Adapter. (a) illustrates a typical architecture of vision Mamba layer [32], including SS2D block, FFN [52] and two residual connections; (b) details of SS2D layer, which transforms the 2D image into sequences and process by SSM equation; (c) recursive solving procedure of SSM with two main steps; (d) Mamba-Adapter with Adaptor-T and Adaptor-S in solving SSM process. FFN and LN denote feed-forward network [53] and layer normalization [71].

**Sequence Transform for Vision Mamba.** Vision Mamba methods [24, 32, 78] has to convert the 2D image data  $x \in \mathbb{R}^{H \times W}$  into the sequential format  $x_s \in \mathbb{R}^{L=H \times W}$  in advance. To gain global context, bidirectional scanning is the basic strategy, while multiple scanning routes are encouraged to obtain more through context modeling ability. For multiple scanning sequences, SSM processes each of them independently and merges the outputs in the final. Here we name the sequence transform and SSM solving procedure in vision Mamba as SS2D scheme.

### 3.2. Formulation of Mamba-Adapter

Our Mamba-Adapter directly functions at the solving Equation 3 and Equation 4 of SSM, which aims to enhance the modeling capability inherently. We explore the potential limitations of the two main steps, i.e., hidden state computation in Equation 3 and output computation in Equation 4 of solving SSM.

**Hidden State.** Each hidden state  $h_i$  for index  $i$  embedding in Equation 3 is a summation of the previous hidden state  $h_{t-1}$  and the transformed value of current input  $u_t$ . The hidden states are computed recursively, beginning with an empty state  $h_0$ . Thus, the far-away hidden state is inaccessible to the current hidden state  $h_t$  when solving Equation 3, resulting in a long-range forgetting issue. Being aware of this issue, a weighted aggregation of hidden states from the non-neighboring area can effectively mitigate the forgetting:

$$h_t = h_i + \sum_{\forall j \in \Lambda} (w_j h_j), \quad (5)$$

where  $\Lambda$  is an index set of selected accessible hidden states and  $w$  is the corresponding weight parameters for summation. Equation 5 formulates the core principle of Adaptor-T, which handles the temporal decay issue in the hidden state of Mamba.

**Final Output.** In Equation 4, the final transformed output  $y_t$  is obtained by a summation of transformed hidden state  $h_t$  and corresponding input embedding  $u_t$ . Then, the final sequential output  $\{y_0, y_1, \dots, y_L\}$  is reshaped to 2D format to obtain the final output  $y \in \mathbb{R}^{H \times W}$  for the next layer. Though the  $h_t$  contains the long-range dependencies through recursive hidden state computation, the spatial locality in 2D image data is largely overlooked in the final output. To overcome this issue, we function at the reshaped 2D format output  $y$  to obtain the spatial dependencies through a Convolutional-style aggregation, which can introduce the image inductive bias effectively:

$$y := \text{Conv}(y), \quad (6)$$

where Conv operator aggregates the weighted output  $y$  in a fixed scanning local window. For each output  $y_{(i,j)}$  in 2D spatial coordinate  $(i, j)$ , the spatial locality is enhanced as follows:

$$y_{(i,j)} = \sum_{\forall (i,j) \in \Omega} w_{(i,j)} y_{(i,j)}, \quad (7)$$

where  $\Omega$  is generally  $K \times K$  local neighbouring embeddings and  $w$  is the corresponding weight parameters. The local adjacency can be enlarged by increasing the kernel size  $K$ . Equation 6 and Equation 7 formulate the core scheme of our Adaptor-S, which functions at the spatial locality of the output in Mamba.

### 3.3. Adaptor-T

The aim of Adaptor-S is to conduct memory retention for each hidden state  $h_i$ . Since Equation 3 is casually computed, each hidden state only has access to the previous states, whose sequential order is smaller than the current one. Brutally aggregating all the previous states may not work well since the forgetting degrees of previous hidden states vary. A fixed handcrafted pattern to select neighboring hidden states, which acts like convolutional filters,

---

**Algorithm 1** Pseudo code of Mamba-Adaptor

---

```




---



```

is also insufficient, as the hidden state that decays may be largely overlooked.

**Learnable Memory Selection.** To handle this, we propose a learnable memory selection scheme, which uses an extremely lightweight prediction layer to select a set of most forgetting hidden states. We adopt an extremely lightweight linear layer  $\phi_p(\cdot)$  to predict  $K$  coordinates  $p_k$  to select the forgetting states for each hidden state  $h_i$ . The coefficient of each forgetting state  $c_k$  is also predicted by a linear layer  $\phi_c(\cdot)$  with a SoftMax function [52]:

$$\begin{aligned} \{p_0, p_1, \dots, p_k\} &= \phi_p(h_i), \\ \{c_0, c_1, \dots, c_k\} &= \text{SoftMax}(\phi_c(h_i)), \end{aligned} \quad (8)$$

Therefore, the memory states most likely to be forgotten are dynamically determined, eliminating the need for manual selection.

**Multi-Sequence Temporal Retention.** Similar to the Multi-head attention scheme [7] and multi-group in convolution filters, the total  $S$  sequences are separately predicted and aggregated:

$$\begin{aligned} h_i^s &:= \sum_{k=1}^K c_{sk} h_s(p_k), s = 1, 2, \dots, S \\ w.r.t \quad \sum_{k=1}^K c_{sk} &= 1. \end{aligned} \quad (9)$$

For the bidirectional scanning [78], the total number of the sequence  $S$  is 2, which needs to predict  $2K$  coordinates. Thus, each hidden state in SSM is strengthened by the predicted most forgetting states, similar to the memory retention scheme in language processing.

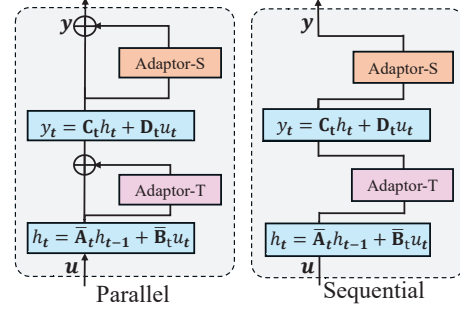


Figure 4. Illustration of the parallel and sequential insertion form of Mamba-Adaptor.

### 3.4. Adaptor-S

Guided by the principle of Adaptor-S, we adopt depthwise convolution filters to aggregate the output features  $y$  from the 2-dimensional spatial aspect. The depthwise convolution is lightweight, bringing acceptable computation overhead. It aggregates the features channel-wise, thereby keeping the shape of output features unchanged.

**Multi-Scale Spatial Aggregation.** Specifically, to enlarge the spatial dependencies, we use multiple depth-wise convolutional filters with multi-scale dilation factors. The spatial locality strengthened for final output feature embedding  $y_{(i,j)}$  in spatial coordinate  $(i,j)$  can be expressed as follows:

$$y_{(i,j)} = \sum_{\forall d} \sum_{\forall (i,j) \in \Omega_d} w_{(i,j)}^d y_{(i,j)}, \quad (10)$$

where  $\Omega_d$  is the index selected from the neighboring area of the depthwise convolutional filter  $d$ . This spatial aggregation process in Adaptor-S brings more structural inductive bias to the sequential feature representations, greatly enhancing the spatial locality in Mamba.

### 3.5. Implementation

We describe how to implement the Mamba-Adaptor using highly optimized hardware modules and introduce two integrated methods for inserting adaptors tailored to different task settings.

**Practical Implementation.** Integrating the Mamba-Adaptor into the highly optimized SSM implementation is essential for effectiveness. We separate the solving procedure into two parts: an original, optimized Mamba operator and highly efficient matrix multiplications. Our Adaptor-T and Adaptor-S can be easily integrated into the Mamba layer. Concretely, we manually generate an identity matrix and zero matrix for  $C$  and  $D$ , respectively, to replace the original parameter. Therefore, the highly optimized Mamba solver [13] can compute the hidden states instead of the final output, which is where Adaptor-T can be inserted. Then, we use matrix multiplications to calculate the final output with

Adaptor-S to enhance the spatial modeling. The overall procedure can be described in the pseudo-code in Alg. 1.

**Layer Insertion** As illustrated in Figure 4, we introduce two forms to integrate our adaptor with vision Mamba models, i.e., sequential and parallel insertion. For image classification and dense prediction tasks that are trained from scratch, we apply sequential form. For transfer learning which finetunes a pretrained model, we adopt the parallel form. The parallel design preserves the original pretrained features through an independent branch, which is better suited for fine-tuning in transfer learning tasks.

**Weight-Sharing Coefficient.** For transfer learning and booster settings, which need a zero weight initialization and less fine-tuned parameters to achieve high efficiency, we adopt a weight-sharing coefficient strategy for Adaptor-S. Concretely, we initialize fixed-shape parameters for coefficients  $\{c_0, c_1, \dots, c_k\}$  for each hidden state, thus reducing the model cost of linear layer  $\phi_c$ .

## 4. Experiment

We evaluate the effectiveness of the Mamba-Adaptor by three major experimental settings. The experiment in Section 4.1 assesses the performance of the visual backbone network for both image classification and dense prediction tasks. The experiment in Section 4.2 involves using the Mamba-Adaptor as a booster module to enhance the performance of pre-trained networks. The experiment in Section 4.3 focuses on transfer learning, where the Mamba-Adaptor acts as an efficient fine-tuning module for downstream recognition tasks. In the final, we depict the ablation studies in Section 4.4.

### 4.1. Visual Backbone Network

**Model Architecture.** To serve as a visual backbone network, Mamba-Adaptor follows a widely-adopted hierarchical backbone architecture [17, 23, 32, 33, 55, 70]. It primarily consists of a patch embedding layer, a vision mamba layer featuring the proposed Mamba-Adaptor to form four model stages, and a head network for classification and dense prediction. Each model stage includes a downsampling layer that reduces the size of feature maps by half while increasing their channel dimension twofold. In this experiment, we introduce two variants of the Mamba-Adaptor, which are as follows:

Mamba-Adaptor-b1 with channel dimension  $\text{nof}$  48 and layer number  $\{2, 2, 6, 2\}$  in four model stage; Mamba-Adaptor-b2 with channel dimension of 96 and layer number  $\{2, 2, 8, 2\}$  in four model stage.

**Image Classification on ImageNet-1K.** The ImageNet-1K [43] dataset comprises 1.28 million images designated for training and 50K images allocated for validation. By using the same training settings [32, 33], we train two variants

| Method             | Params (M) | FLOPs (G) | Top1-acc (%) |
|--------------------|------------|-----------|--------------|
| RegNetY-1.6G       | 11         | 1.6       | 78.0         |
| EfficientNet-B3    | 12         | 1.8       | 81.6         |
| PVTv2-b1 [56]      | 13         | 2.1       | 78.7         |
| BiFormer-T [77]    | 13         | 2.2       | 81.4         |
| Vim-T              | 7          | 1.5       | 76.1         |
| LocalViM-T         | 8          | 1.5       | 76.2         |
| Mamba-Adaptor-b1   | 7.8        | 1.4       | 78.4         |
| CoAtNet-T [35]     | 29         | 4.5       | 82.1         |
| UniRepLkNet-T [35] | 29         | 4.5       | 82.1         |
| ConvNeXt-T [35]    | 29         | 4.5       | 82.1         |
| MambaOut-T         | 27         | 4.5       | 82.7         |
| InternImage-T [58] | 30         | 5.0       | 83.5         |
| DeiT-S [51]        | 22         | 4.6       | 79.9         |
| Swin-T [33]        | 29         | 4.5       | 81.3         |
| Vim-S              | 26         | 5.1       | 80.3         |
| VMamba-T           | 22         | 5.6       | 82.6         |
| LocalVMamba        | 26         | 5.7       | 82.7         |
| Mamba-Adaptor-b2   | 32         | 5.4       | 83.0         |

Table 1. Comparison of state-of-the-art methods for ImageNet-1K [43] classification.

of Mamba-Adaptor from scratch using the training set and evaluate their performance by reporting the Top-1 accuracy on the validation set. We assessed VMamba’s performance in image classification on the ImageNet-1K dataset, and the results comparing it with benchmark methods are presented in Table 1. With similar computation FLOPs, our Mamba-Adaptor-b1 achieves a top-1 accuracy of 78.4, surpassing LocalViM-T by 2.2 % and Vim-T by 2.3 %. Moreover, Mamba-Adaptor-b2 also retains its performance advantage at larger scales. For example, Mamba-Adaptor-b2 achieves a top-1 accuracy of 82.9, exceeding Swin-T by 2.6 % and VMamba-T by 0.2%.

**Object Detection on COCO.** COCO [30] is an object detection and instance segmentation dataset that comprises 118K training images and 5K validation images. We utilize Mamba-Adaptor as the backbone in MaskRCNN detection framework [18] to assess the effectiveness of our method. We follow similar training strategies in SwinT [33], which use the pretrained weights on the ImageNet-1K dataset. The results on MSCOCO are shown in Table 2. Our Mamba-Adaptor demonstrates superiority in both box and mask Average Precision ( $AP^b$  and  $AP^m$ ), in both “1×” and “3×” training schedules. With the “1×” fine-tuning schedule and similar computational FLOPs, Mamba-Adaptor-b1 achieves best object detection mAPs of 43.2%, surpassing EffVMamba-S [41] by 3.9 %  $AP^b$ , and PVT-T by 6.5 %  $AP^m$ , respectively. Under the fair comparisons, the instance segmentation mAPs achieved by Mamba-Adaptor-b2 outperform the baseline VMamba-T by 1.8% in  $AP^b$  and 2.3 in  $AP^m$ . Mamba-Adaptor-b2 also shows advantages to the improved vision Mamba model, LocalVMamba-T in both object detection and instance segmentation tasks. The Mamba-Adaptor demonstrates its potential to deliver im-

| Backbone         | Params<br>(M) | FLOPs<br>(G) | Mask R-CNN 1×+MS |             |             |        |             |             | Mask R-CNN 3×+MS |             |             |        |             |             |
|------------------|---------------|--------------|------------------|-------------|-------------|--------|-------------|-------------|------------------|-------------|-------------|--------|-------------|-------------|
|                  |               |              | $AP^b$           | $AP_{50}^b$ | $AP_{75}^b$ | $AP^m$ | $AP_{50}^m$ | $AP_{75}^m$ | $AP^b$           | $AP_{50}^b$ | $AP_{75}^b$ | $AP^m$ | $AP_{50}^m$ | $AP_{75}^m$ |
| ResNet18         | 21.3          | 189          | 31.8             | 49.6        | 33.6        | 16.3   | 34.3        | 43.2        | -                | -           | -           | -      | -           | -           |
| PVT-T            | 33            | 208          | 36.7             | 59.2        | 39.3        | 35.1   | 56.7        | 37.3        | 39.8             | 62.2        | 43.0        | 37.4   | 59.3        | 39.9        |
| EffVMamba-S      | 31            | 197          | 39.3             | 61.8        | 42.8        | 36.7   | 58.9        | 39.2        | 41.6             | 63.9        | 45.6        | 38.2   | 60.8        | 40.7        |
| Mamba-Adaptor-b1 | 32            | 218          | 43.2             | 65.5        | 47.7        | 39.5   | 60.1        | 42.7        | 45.1             | 67.2        | 49.4        | 41.2   | 61.9        | 43.8        |
| ResNet-50        | 44            | 260          | 38.2             | 58.8        | 41.4        | 34.7   | 55.7        | 37.2        | -                | -           | -           | -      | -           | -           |
| Swin-T           | 48            | 267          | 42.7             | 65.2        | 46.8        | 39.3   | 62.2        | 42.2        | 46.0             | 68.1        | 50.3        | 41.6   | 65.1        | 44.9        |
| VMamba-T         | 50            | 271          | 46.5             | 68.5        | 52.0        | 42.1   | 65.5        | 45.3        | 48.8             | 70.4        | 53.5        | 43.7   | 67.4        | 47.0        |
| LocalVMamba-T    | 45            | 291          | 46.7             | 68.7        | 50.8        | 42.2   | 65.7        | 45.5        | 48.7             | 70.1        | 53.0        | 43.4   | 67.0        | 46.4        |
| Mamba-Adaptor-b2 | 42            | 259          | 47.3             | 69.8        | 52.3        | 43.4   | 66.9        | 46.4        | 49.1             | 71.5        | 54.1        | 44.8   | 67.3        | 48.3        |

Table 2. Comparison to the state-of-the-art backbone networks using Mask R-CNN with "1×" and "3×" training schedules.

| Method           | Params<br>(M) | FLOPs<br>(G)  | Top1-acc<br>(%) |
|------------------|---------------|---------------|-----------------|
| VMamba-T         | 30            | 4.9           | 82.6            |
| Adaptor-VMamba-T | 31 (3.2% ↑)   | 5.2 (6.1% ↑)  | <b>82.7</b>     |
| VMamba-S         | 50            | 8.7           | 83.6            |
| Adaptor-VMamba-S | 53 (6.1% ↑)   | 9.3 (8.0% ↑)  | <b>83.7</b>     |
| VMamba-B         | 89            | 15.4          | 83.9            |
| Adaptor-VMamba-B | 94 (5.6% ↑)   | 16.5 (7.1% ↑) | <b>84.1</b>     |

Table 3. Improvements over ImageNet-1K classification. Adaptor-Baseline denotes the baseline model, which is equipped with our Mamba-Adaptor as a booster module.

pressive performance in downstream tasks related to dense prediction.

## 4.2. Booster Network for Image Recognition

**Booster, Baseline, and Benchmark.** We choose VMamba-T/S/B pretrained in ImageNet [43] classification as the baseline model and equip it with the proposed Mamba-Adaptor with acceptable computation overhead. We are continuing to fine-tune the VMamba equipped with the Mamba-Adaptor on the ImageNet training set for an additional 10 epochs.

**Results on Booster.** The results in Table 3 show that our Mamba-Adaptor can raise the performance of the pretrained VMamba [32] baseline further. Concretely, with less than 3.2% extra model parameters and 6.1 % FLOPS, Mamba-Adaptor can raise the VMamba-T by 0.1% top1 accuracy. With the VMamba-B with a large model size, the boosting performance is increased to 0.2 % in top1 accuracy. It is worth noting that the booster training only requires 10 additional epochs, which indicates that our Mamba-Adaptor as a booster module is highly efficient.

## 4.3. Adaptor Network for Transfer Learning

**Adaptor, Baseline, and Benchmark.** We compare Mamba-Adaptor with three commonly used methods for fine-tuning. (1) Linear probing: it involves adding an extra linear layer on top of the backbone and tuning the linear layer for evaluation. (2) Full Fine-tuning: we unfreeze all model parameters and train them together. (3) Visual

Prompt Tuning (VPT) [25]: fine-tuning the additional visual tokens as prompt (we add 128 extra tokens). We choose the VMamba [32] with three variants, i.e., tiny/small/base, to evaluate the performance of the Adaptor. We also present the ViT-Base [7] baseline network for comparison in Figure 4. Following the setting in [2, 25], we choose three benchmarks for image recognition: CIFAR-100 [26] includes 50,000 training images and 10K validation images at a resolution of 32×32 pixels across 100 labels; The Street View House Numbers (SVHN) [11] dataset, used for digit classification, consists of over 600K labeled images, with 73K for training, 26K for testing, and 531K extra training images; The Food-101 [1] dataset features 101 food categories, totaling 101K images.

**Results on Transfer Learning.** The results on three visual recognition benchmarks are reported in Table 3. Mamba-Adaptor maintains superior performance in top 1 classification accuracy across different datasets, compared to the other two finetuning methods, i.e., linear probe and VPT [25]. Specifically, for the CIFAR-100 benchmark, Mamba-Adaptor-T/S/B preserves a 90% performance with less than 5.56%/9.25%/7.13% model parameters, compared to the full finetuning method, which incorporates all the model parameters into the training. The performance of the linear probe and VPT [25] methods has a large gap between the full fine-tuning in all three visual recognition benchmarks [1, 11, 26], validating the effectiveness of our method.

## 4.4. Ablation Studies

**Adaptor-T/S on Classification and Prediction tasks.** Table 6 shows the ablation studies on the two functional modules with different configurations, i.e., static/learnable state selection for Adaptor-T and multi-scale factor for the Adaptor-S. Compared to the base model without Mamba-Adaptor (①), all the remaining settings (② to ⑥) show improvement in varying degrees in both classification and dense prediction tasks. Notably, the learnable state selection scheme surpasses the static selection scheme with 0.3% top-1 accuracy in ImageNet1k and 0.6% in  $AP^b$  in COCO object detection, validating the effectiveness of our dynamic

| Adaptor Method | Base Model    | Pretrain  | Paras (M)    | CIFAR-100 (Acc%) | SVHN (Acc%)    | Food101 (Acc%) |
|----------------|---------------|-----------|--------------|------------------|----------------|----------------|
| Full-Tuning    | ViT-B [7]     | MAE [19]  | 86.04 (100%) | 85.90            | 97.67          | 90.09          |
| Linear         | ViT-B [7]     | MAE [19]  | 0.07 (0.08%) | 69.83 (-16.07)   | 66.91 (-30.76) | 69.74 (-20.35) |
| VPT [25]       | ViT-B [7]     | MAE [19]  | 0.08 (0.09%) | 82.44 (-3.46)    | 94.02 (-3.65)  | 82.98 (-7.11)  |
| Full-Tuning    | VMamba-T [32] | Cls. [32] | 30.25 (100%) | 87.48            | 97.82          | 90.22          |
| Linear         | VMamba-T [32] | Cls. [32] | 0.02 (0.06%) | 61.23 (-26.25)   | 54.36 (-43.46) | 61.62 (-28.60) |
| VPT [25]       | VMamba-T [32] | Cls. [32] | 0.03 (0.09%) | 80.68 (-6.80)    | 89.23 (-8.59)  | 80.34 (-9.78)  |
| Mamba-Adaptor  | VMamba-T [32] | Cls. [32] | 1.68 (5.56%) | 86.82 (-0.66)    | 93.24 (-4.58)  | 86.45 (-3.76)  |
| Full-Tuning    | VMamba-S [32] | Cls. [32] | 55.25 (100%) | 89.59            | 97.90          | 91.24          |
| Linear         | VMamba-S [32] | Cls. [32] | 0.02 (0.04%) | 65.34 (-24.25)   | 52.12 (-45.78) | 68.98 (-22.26) |
| VPT [25]       | VMamba-S [32] | Cls. [32] | 0.03 (0.05%) | 82.26 (-6.33)    | 82.48 (-15.42) | 64.23 (-27.01) |
| Mamba-Adaptor  | VMamba-S [32] | Cls. [32] | 5.10 (9.25%) | 88.30 (-1.39)    | 85.69 (-12.21) | 82.33 (-8.91)  |
| Full-Tuning    | VMamba-B [32] | Cls. [32] | 95.36 (100%) | 89.89            | 97.96          | 91.68          |
| Linear         | VMamba-B [32] | Cls. [32] | 0.03 (0.03%) | 67.23 (-22.66)   | 55.69 (-42.27) | 69.42 (-22.16) |
| VPT [25]       | VMamba-B [32] | Cls. [32] | 0.04 (0.04%) | 81.32 (-8.57)    | 87.45 (-10.51) | 80.34 (-11.36) |
| Mamba-Adaptor  | VMamba-B [32] | Cls. [32] | 6.8 (7.13%)  | 88.34 (-1.5)     | 88.52 (-8.44)  | 83.21 (-8.47)  |

Table 4. Fine-tuning with the pre-trained base models for transfer learning tasks, where Mamba-Adaptor serves as an adaptor network. For tunable parameters, we report the percentage of the parameters. Additionally, we show both the absolute value and the gap value relative to the top 1 accuracy of the full-tuning setting. Cls. denotes normal image classification training [32, 33].

| Setting | VMamba-T  | VMamba-S | Top1-acc (%) | $AP^b$ (%)  | $AP^m$ (%)  |
|---------|-----------|----------|--------------|-------------|-------------|
| ①       | ✗         | ✗        | 77.2         | 41.2        | 42.4        |
| ②       | static    | ✗        | 77.6         | 41.6        | 42.9        |
| ③       | ✗         | 1 scale  | 77.5         | 41.4        | 42.7        |
| ④       | static    | 1 scale  | 77.9         | 42.3        | 43.9        |
| ⑤       | learnable | 1 scale  | 78.2         | 42.9        | 44.4        |
| ⑥       | learnable | 2 scales | <b>78.4</b>  | <b>43.2</b> | <b>44.8</b> |

Table 5. Ablation study on the two functional modules of Mamba-Adaptor. The Mamba-Adaptor-b1 variant is adopted.

| Setting | Init.  | Insert.    | Weight S. | CIFAR-100    | SVHN         | Food101      |
|---------|--------|------------|-----------|--------------|--------------|--------------|
| ①       | random | sequential | ✗         | 62.34        | 54.32        | 65.12        |
| ②       | zero   | parallel   | ✗         | 77.23        | 67.98        | 78.32        |
| ③       | zero   | sequential | ✗         | 85.33        | 92.43        | 83.43        |
| ④       | zero   | parallel   | ✓         | <b>86.82</b> | <b>93.24</b> | <b>86.45</b> |

Table 6. Ablation study on different configurations for transfer learning. Weight S. denotes the weight-sharing coefficient in Adaptor-T. Init. denotes the initialization method for the linear layer in Adaptor-T/S.

selection scheme. We also observe a performance gain (0.7% in  $AP^m$ ) when adding additional depthwise kernels to the Adaptor-S due to the hierarchical representations.

**On Transfer Learning.** As shown in Table 6, the initialization method matters for the transfer learning tasks. The zero initialization significantly raises the performance by 14.9/13.6/13.2 % in top1 accuracy (② vs. ①) in all three benchmarks compared to the random initialization. The parallel insertion form also contributes significantly to the performance in the transfer learning setting, as can be validated by the comparison between ② and ③. The potential reason is that zero initialization and the parallel insertion design maintain the original feature. It also matters for the booster module design as the pre-trained weight contains

| Transfer learning | CIFAR | SVNH  | Food101 |
|-------------------|-------|-------|---------|
| ViM-Adaptor       | +8.31 | +6.27 | +9.18   |
| VMamba-Bi-Adaptor | +6.42 | +4.97 | +7.02   |

Table 7. Transfer learning on Vision Mamba with bi-directional scanning, especially in comparison to the VPT [25] method.

large-scale prior knowledge of the ImageNet [43], and the extra Mamba-Adaptor module is of small model parameters. The weight-sharing coefficient strategy for Adaptor-T also improves the final performance while reducing the increased model parameters.

**On Generalization ability.** We also conducted transfer learning experiments to show generalization ability. Tab. 7 shows that our method also shows good results on the Vision Mamba with bi-directional scanning [31].

## 5. Conclusion and Limitations

This paper presents a plug-and-play Mamba-Adaptor, which has three major usages. Mamba-Adaptor can serve as a general visual backbone for image classification and dense prediction tasks, which improves the baseline considerably. Moreover, it can also serve as a booster module that is inserted into a pre-trained vision backbone to enhance performance through extended training. In transfer learning scenarios, the Mamba-Adaptor is an efficient module that enables the adaptation of a pre-trained Mamba backbone for various downstream visual recognition tasks. Extensive experimental results evaluate the effectiveness of the Mamba-Adaptor. We hope that Mamba-Adaptor offers an insightful solution to enhance the Mamba scheme in the field of vision. However, the scaling of the Mamba-Adaptor as a visual backbone remains open, and its applications to other variants of vision Mamba are part of future work.

## References

- [1] Lukas Bossard, Matthieu Guillaumin, and Luc Van Gool. Food-101—mining discriminative components with random forests. In *ECCV*, 2014. 7
- [2] Shoufa Chen, Chongjian Ge, Zhan Tong, Jiangliu Wang, Yibing Song, Jue Wang, and Ping Luo. Adaptformer: Adapting vision transformers for scalable visual recognition. In *NeurIPS*, 2022. 3, 7
- [3] Zhe Chen, Yuchen Duan, Wenhai Wang, Junjun He, Tong Lu, Jifeng Dai, and Yu Qiao. Vision transformer adapter for dense predictions. In *ICLR*, 2023. 3
- [4] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Annual Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 4171–4186, 2019. 3
- [5] Mingyu Ding, Bin Xiao, Noel Codella, et al. Davit: Dual attention vision transformers. In *ECCV*, 2022. 2
- [6] Wenhao Dong, Haodong Zhu, Shaohui Lin, Xiaoyan Luo, Yunhang Shen, Xuhui Liu, Juan Zhang, Guodong Guo, and Baochang Zhang. Fusion-mamba for cross-modality object detection, 2024. 1
- [7] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. An image is worth 16x16 words: Transformers for image recognition at scale. In *ICLR*, 2020. 1, 3, 5, 7, 8
- [8] Zhengcong Fei, Mingyuan Fan, Changqian Yu, Debang Li, Youqiang Zhang, and Junshi Huang. Dimba: Transformer-mamba diffusion models. *arXiv preprint*, 2024. 1
- [9] Peng Gao, Shijie Geng, Renrui Zhang, Teli Ma, Rongyao Fang, Yongfeng Zhang, Hongsheng Li, and Yu Qiao. Clip-adapter: Better vision-language models with feature adapters. *IJCV*, 2024. 3
- [10] Ross Girshick. Fast R-cnn. In *Proceedings of International Conference on Computer Vision*, 2015. 2
- [11] Ian J Goodfellow, Yaroslav Bulatov, Julian Ibarz, Sacha Arnold, and Vinay Shet. Multi-digit number recognition from street view imagery using deep convolutional neural networks. *arXiv preprint*, 2013. 7
- [12] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint*, 2023. 3
- [13] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint*, 2023. 1, 2, 3, 5
- [14] Albert Gu, Tri Dao, Stefano Ermon, Atri Rudra, and Christopher Ré. Hippo: Recurrent memory with optimal polynomial projections. *NeurIPS*, 2020. 2
- [15] Albert Gu, Karan Goel, and Christopher Ré. Efficiently modeling long sequences with structured state spaces. In *ICLR*, 2022. 1, 2, 3
- [16] Ali Hatamizadeh and Jan Kautz. Mambavision: A hybrid mamba-transformer vision backbone. *arXiv preprint*, 2024. 2
- [17] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *CVPR*, 2016. 2, 6
- [18] Kaiming He, Georgia Gkioxari, Piotr Dollár, and Ross Girshick. Mask r-cnn. In *ICCV*, 2017. 2, 6
- [19] Kaiming He, Xinlei Chen, Saining Xie, Yanghao Li, Piotr Dollár, and Ross Girshick. Masked autoencoders are scalable vision learners. In *CVPR*, 2022. 8
- [20] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997. 1
- [21] Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin De Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. Parameter-efficient transfer learning for nlp. In *ICML*, 2019. 3
- [22] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. In *ICLR*, 2022. 3
- [23] Gao Huang, Zhuang Liu, Laurens Van Der Maaten, and Kilian Q Weinberger. Densely connected convolutional networks. In *CVPR*, 2017. 6
- [24] Tao Huang, Xiaohuan Pei, Shan You, Fei Wang, Chen Qian, and Chang Xu. Localmamba: Visual state space model with windowed selective scan. *arXiv preprint*, 2024. 1, 2, 4
- [25] Menglin Jia, Luming Tang, Bor-Chun Chen, Claire Cardie, Serge Belongie, Bharath Hariharan, and Ser-Nam Lim. Visual prompt tuning. In *ECCV*, 2022. 3, 7, 8
- [26] Alex Krizhevsky, Geoffrey Hinton, et al. Learning multiple layers of features from tiny images. *arXiv preprint*, 2009. 7
- [27] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. Imagenet classification with deep convolutional neural networks. In *NeurIPS*, 2012. 2
- [28] Shufan Li, Harkanwar Singh, and Aditya Grover. Mamband: Selective state space modeling for multi-dimensional data. *arXiv preprint*, 2024. 3
- [29] Yanghao Li, Hanzi Mao, Ross Girshick, and Kaiming He. Exploring plain vision transformer backbones for object detection. In *ECCV*, 2022. 3
- [30] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C Lawrence Zitnick. Microsoft coco: Common objects in context. In *ECCV*, 2014. 1, 6
- [31] Yue Liu, Yunjie Tian, Yuzhong Zhao, Hongtian Yu, Lingxi Xie, Yaowei Wang, Qixiang Ye, and Yunfan Liu. Vmamba: Visual state space model. *Neurips*, 2024. 8
- [32] Yue Liu, Yunjie Tian, Yuzhong Zhao, Hongtian Yu, Lingxi Xie, Yaowei Wang, Qixiang Ye, and Yunfan Liu. Vmamba: Visual state space model. *NeurIPS*, 2024. 1, 2, 4, 6, 7, 8
- [33] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. Swin transformer: Hierarchical vision transformer using shifted windows. In *CVPR*, 2021. 1, 2, 6, 8
- [34] Ze Liu, Han Hu, Yutong Lin, Zhuliang Yao, Zhenda Xie, Yixuan Wei, Jia Ning, Yue Cao, Zheng Zhang, Li Dong, et al. Swin transformer v2: Scaling up capacity and resolution. In *CVPR*, 2022. 3

- [35] Zhuang Liu, Hanzi Mao, Chao-Yuan Wu, et al. A convnet for the 2020s. In *CVPR*, 2022. 6
- [36] Jun Ma, Feifei Li, and Bo Wang. U-mamba: Enhancing long-range dependency for biomedical image segmentation. *arXiv preprint*, 2024. 1, 2
- [37] Harsh Mehta, Ankit Gupta, Ashok Cutkosky, and Behnam Neyshabur. Long range language modeling via gated state spaces. In *ICLR*, 2023. 2
- [38] Eric Nguyen, Karan Goel, Albert Gu, Gordon W Downs, Preeti Shah, Tri Dao, Stephen A Baccus, and Christopher Ré. S4nd: Modeling images and videos as multidimensional signals using state spaces. *arXiv preprint*, 2022. 1, 2
- [39] Jiahao Nie, Zhiwei He, Xudong Lv, Xueyi Zhou, Dong-Kyu Chae, and Fei Xie. Towards category unification of 3d single object tracking on point clouds. In *ICLR*, 2024. 1
- [40] Adam Paszke, Sam Gross, Soumith Chintala, Gregory Chanan, Edward Yang, Zachary DeVito, Zeming Lin, Alban Desmaison, Luca Antiga, and Adam Lerer. Automatic differentiation in PyTorch. In *NeurIPS Workshop*, 2017. 2
- [41] Xiaohuan Pei, Tao Huang, and Chang Xu. Efficientv-mamba: Atrous selective scan for light weight visual mamba. *NeurIPS*, 2024. 2, 6
- [42] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, et al. Learning transferable visual models from natural language supervision. In *ICML*, 2021. 3
- [43] Olga Russakovsky, Jia Deng, Hao Su, Jonathan Krause, Sanjeev Satheesh, Sean Ma, Zhiheng Huang, Andrej Karpathy, Aditya Khosla, and Michael Bernstein. ImageNet Large scale visual recognition challenge. *IJCV*, 2015. 1, 6, 7, 8
- [44] Jimmy TH Smith, Andrew Warrington, and Scott W Linderman. Simplified state space layers for sequence modeling. *arXiv preprint*, 2022. 2, 3
- [45] Lin Song, Ruoyi Xue, Hang Wang, Hongbin Sun, Yixiao Ge, Ying Shan, et al. Meta-adapter: An online few-shot learner for vision-language model. In *NeurIPS*, 2023. 3
- [46] Asa Cooper Stickland and Iain Murray. Bert and pals: Projected attention layers for efficient adaptation in multi-task learning. In *ICML*, 2019. 3
- [47] Yi-Lin Sung, Jaemin Cho, and Mohit Bansal. V1-adapter: Parameter-efficient transfer learning for vision-and-language tasks. In *CVPR*, 2022. 3
- [48] Christian Szegedy, Wei Liu, Yangqing Jia, Pierre Sermanet, Scott Reed, Dragomir Anguelov, Dumitru Erhan, Vincent Vanhoucke, and Andrew Rabinovich. Going deeper with convolutions. In *CVPR*, 2015. 2
- [49] Yujin Tang, Lu Qi, Fei Xie, Xiangtai Li, Chao Ma, and Ming-Hsuan Yang. Predformer: Transformers are effective spatial-temporal predictive learners. *arXiv:2410.04733*, 2024. 1
- [50] Yao Teng, Yue Wu, Han Shi, Xuefei Ning, Guohao Dai, Yu Wang, Zhenguo Li, and Xihui Liu. Dim: Diffusion mamba for efficient high-resolution image synthesis. *arXiv preprint arXiv:2405.14224*, 2024. 1
- [51] Hugo Touvron, Matthieu Cord, Matthijs Douze, et al. Training data-efficient image transformers & distillation through attention. In *ICML*, 2021. 2, 6
- [52] Ashish Vaswani, Noam Shazeer, Niki Parmar, et al. Attention is all you need. In *NeurIPS*, 2017. 3, 4, 5
- [53] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *NeurIPS*, 2020. 1, 4
- [54] Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. GLUE: A multi-task benchmark and analysis platform for natural language understanding. In *ICLR*, 2019. 2
- [55] Wenhai Wang, Enze Xie, Xiang Li, Deng-Ping Fan, Kaitao Song, Ding Liang, Tong Lu, Ping Luo, and Ling Shao. Pyramid vision transformer: or dense prediction without convolutions. In *ICCV*, 2021. 1, 2, 6
- [56] Wenhai Wang, Enze Xie, Xiang Li, Deng-Ping Fan, Kaitao Song, Ding Liang, Tong Lu, Ping Luo, and Ling Shao. Pvt2: Improved baselines with pyramid vision transformer. *Computational Visual Media*, 2022. 6
- [57] Wenxiao Wang, Lu Yao, Long Chen, Binbin Lin, Deng Cai, Xiaofei He, and Wei Liu. Crossformer: A versatile vision transformer hinging on cross-scale attention. In *ICLR*, 2022. 2
- [58] Wenhai Wang, Jifeng Dai, Zhe Chen, Zhenhang Huang, Zhiqi Li, Xizhou Zhu, Xiaowei Hu, Tong Lu, Lewei Lu, Hongsheng Li, et al. Internimage: Exploring large-scale vision foundation models with deformable convolutions. In *CVPR*, 2023. 3, 6
- [59] Karl Weiss, Taghi M Khoshgoftaar, and DingDing Wang. A survey of transfer learning. *Journal of Big data*, 3:1–40, 2016. 3
- [60] Robert L Williams, Douglas A Lawrence, et al. *Linear state-space control systems*. John Wiley & Sons, 2007. 2
- [61] Fei Xie, Ning Wang, Yuncong Yao, Wankou Yang, Kaihua Zhang, and Bo Liu. Hierarchical representations with discriminative meta-filters in dual path network for tracking. In *PRCV*, 2020. 1
- [62] Fei Xie, Chunyu Wang, Guangting Wang, Wankou Yang, and Wenjun Zeng. Learning tracking representations via dual-branch fully transformer networks. In *ICCVw*, 2021. 1
- [63] Fei Xie, Wankou Yang, Kaihua Zhang, Bo Liu, Guangting Wang, and Wangmeng Zuo. Learning spatio-appearance memory network for high-performance visual tracking. In *ICCVw*, 2021. 1
- [64] Fei Xie, Chunyu Wang, Guangting Wang, Yue Cao, Wankou Yang, and Wenjun Zeng. Correlation-aware deep tracking. In *Proceedings of IEEE Conference on Computer Vision and Pattern Recognition*, 2022. 1
- [65] Fei Xie, Lei Chu, Jiahao Li, Yan Lu, and Chao Ma. Video-track: Learning to track objects via video transformer. In *CVPR*, 2023. 1
- [66] Fei Xie, Zhongdao Wang, and Chao Ma. Diffusiontrack: Point set diffusion model for visual object tracking. In *CVPR*, 2024. 1
- [67] Fei Xie, Wankou Yang, Chunyu Wang, Lei Chu, Yue Cao, Chao Ma, and Wenjun Zeng. Correlation-embedded transformer tracking: A single-branch framework. *TPAMI*, 2024. 1
- [68] Fei Xie, Weijia Zhang, Zhongdao Wang, and Chao Ma. Quadmambata: Learning quadtree-based selective scan for visual state space model. In *NeurIPS*, 2024. 2

- [69] Saining Xie, Ross Girshick, Piotr Dollár, Zhuowen Tu, and Kaiming He. Aggregated residual transformations for deep neural networks. In *ICCV*, 2017. [2](#)
- [70] Saining Xie, Ross Girshick, Piotr Dollár, Zhuowen Tu, and Kaiming He. Aggregated residual transformations for deep neural networks. In *CVPR*, 2017. [6](#)
- [71] Jingjing Xu, Xu Sun, Zhiyuan Zhang, Guangxiang Zhao, and Junyang Lin. Understanding and improving layer normalization. *NeurIPS*, 32, 2019. [4](#)
- [72] Chenhongyi Yang, Zehui Chen, Miguel Espinosa, Linus Ericsson, Zhenyu Wang, Jiaming Liu, and Elliot J Crowley. Plainmamba: Improving non-hierarchical mamba in visual recognition. *arXiv preprint*, 2024. [1](#), [2](#)
- [73] Weihao Yu, Pan Zhou, Shuicheng Yan, and Xinchao Wang. Inceptionnext: when inception meets convnext. In *CVPR*, 2023. [2](#)
- [74] Renrui Zhang, Rongyao Fang, Wei Zhang, Peng Gao, Kunchang Li, Jifeng Dai, Yu Qiao, and Hongsheng Li. Tip-adapter: Training-free clip-adapter for better vision-language modeling. *arXiv preprint*, 2021. [3](#)
- [75] Wenkang Zhang, Fei Xie, Tianyang Xu, Jiang Zhai, and Wankou Yang. Crtrack: Learning correlation-refine network for visual object tracking. *Pattern Recognition*, page 110582, 2024. [1](#)
- [76] Bolei Zhou, Hang Zhao, Xavier Puig, et al. Scene parsing through ade20k dataset. In *CVPR*, 2017. [1](#)
- [77] Lei Zhu, Xinjiang Wang, Zhanghan Ke, Wayne Zhang, and Rynson Lau. Biformer: Vision transformer with bi-level routing attention. In *CVPR*, 2023. [6](#)
- [78] Lianghui Zhu, Bencheng Liao, Qian Zhang, Xinlong Wang, Wenyu Liu, and Xinggang Wang. Vision mamba: Efficient visual representation learning with bidirectional state space model. *ICML*, 2024. [1](#), [2](#), [4](#), [5](#)